

# DESIGN AND IMPLEMENTATION OF SELF-TEST Using Verilog

NIDHI KATHURIA

A mechanism that allows a machine to test itself is called built-in self-test (or BIST). It can generate patterns based on a variety of algorithms, each focused on a particular type of circuitry or fault type. Comparison function has a number of unique implementations including actual comparators as well as signal analysers.

In this project we will design Memory BIST (MBIST), which uses one or more algorithms specifically designed for testing memory faults. BIST structures generate patterns and compare output responses for a dedicated piece of circuitry. You can implement BIST on entire designs, design blocks or structures within design blocks. Pattern generation as well as output-comparison circuitry can vary depending on the design.

MBIST circuitry generates patterns and detects device failures. A basic MBIST block diagram is shown in Fig. 1.

## BIST fault models

A manufacturing defect is a physical problem that occurs during the manufacturing process, causing device malfunctions of some kind. The purpose of test generation is to create a set of test patterns that detect as many manufacturing defects as possible.

The basic types of memory faults include stuck-at, transition, coupling and neighbourhood pattern-sensitive. In this project we will design BIST for memory stuck-at fault model, which means that

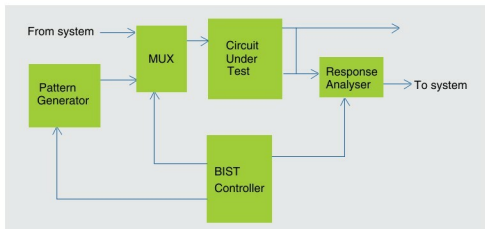


Fig. 1: Circuit with surrounding built-in self-test circuitry

a memory fails if one of its control signals or memory cells remains stuck at a particular value.

Stuck-at faults model this behaviour, where a signal or cell appears to be tied to power (stuck-at-1) or ground (stuck-at-0).

Device testing requires stimulus, a mechanism to apply stimulus to the device or circuit under test (CUT), and some means to analyse or compare the device's responses with a known good (non-faulty) response.

Classical testing uses external test patterns as stimulus and applies the patterns to the device via a tester. The tester examines the device's response, comparing it against the known good response stored as part of the test pattern data.

Fig. 1 shows how BIST places all these functions within the circuitry surrounding the CUT. A state machine is used to generate stimulus and analyse the response of the CUT.

**BIST specifications.** BIST is basically used to help in the testing

of memory, which is an extremely complex architecture (fabrication wise), with the help of a few pins. In fact, while testing a memory using BIST, applying a simple clock signal along with a few pins helps test the entire memory IC.

Presented here is a BIST design using Verilog, which is simulated using ModelSim software.

Here, we design a memory model, BIST controller and its test bench, which is used to drive the entire operation of BIST. In `bist_controller.v` code, the finite-state machine is driven via a data generator, an address generator and a control generator. Also, MUX is used to select the operation of BIST or normal memory operations. (For details, please refer to the source codes of the project included in the accompanying DVD.)

We design and analyse BIST using the pattern generator, which will write and read-back same patterns in all memory locations.

We considered 1024 inputs, each having 32-bit data. Now, for